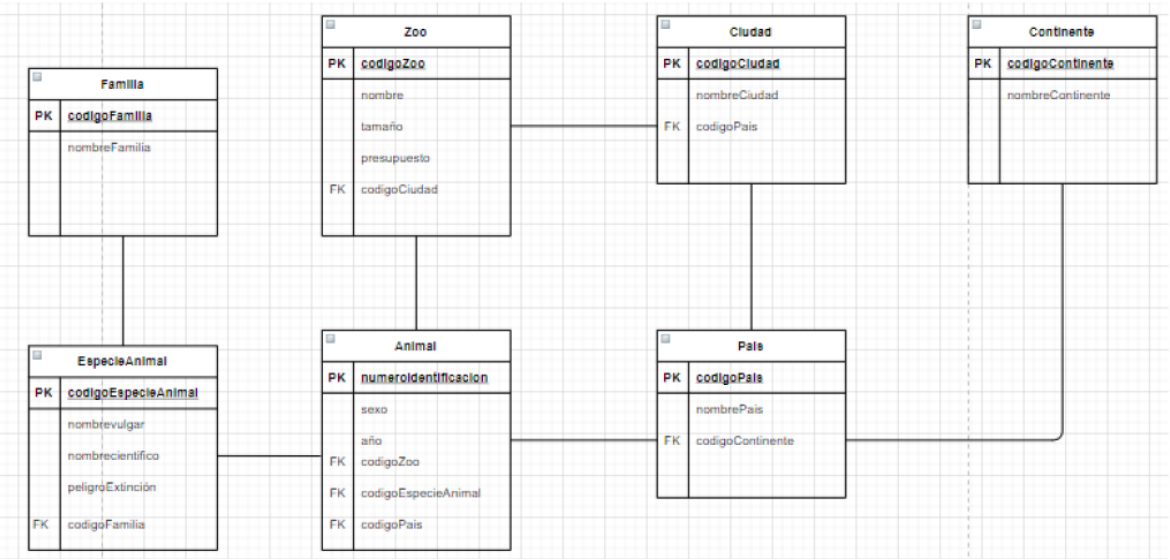
Creación de un modelo de datos físico con script

El modelo relacional o Lógico resultante es:



Una base de datos almacena su información en tablas, que es la unidad básica de almacenamiento.

Una tabla es una estructura de datos que organiza los datos en columnas y filas; cada columna es un campo (o atributo) y cada fila, un registro.

Al crear una tabla debemos resolver qué campos (columnas) tendrá y que tipo de datos almacenarán cada uno de ellos, es decir, su estructura.

La sintaxis básica y general para crear una tabla es la siguiente:

Sintaxi

```
create table NOMBRETABLA(
  NOMBRECAMPO1 TIPODEDATO,
  ...
  NOMBRECAMPON TIPODEDATO
 );
```

Cada campo con su tipo debe separarse con comas de los siguientes, excepto el último.

Nota: Si intentamos crear una tabla con un nombre ya existente (existe otra tabla con ese nombre), mostrará un mensaje indicando que a tal nombre ya lo está utilizando otro objeto y la sentencia no se ejecutará.

Podemos encontrar distintas formas de crear las tablas del modelo. Por ejemplo:

1. Crear la tabla sin llaves

```
CREATE TABLE continente
(
  CODIGOCONTINENTE  VARCHAR2(3 BYTE),
  NOMBRECONTINENTE VARCHAR2(100 BYTE)
);
```

2. Crear la tabla con llave primaria al lado de la columna

```
CREATE TABLE continente
(
  CODIGOCONTINENTE  VARCHAR2(3 BYTE) PRIMARY KEY,
  NOMBRECONTINENTE VARCHAR2(100 BYTE)
);
```

3. Crear la tabla con llave primaria al final del listado de columnas.

```
CREATE TABLE continente
(
  CODIGOCONTINENTE  VARCHAR2(3 BYTE),
  NOMBRECONTINENTE VARCHAR2(100 BYTE),
  PRIMARY KEY (CODIGOCONTINENTE)
);
```

En el segundo y tercer caso, la primary key creada internamente posee un nombre que la base le asocia y no crea restricciones donde la columna del primary key no pueda ser nulo. Ejemplo:

Columnas | Datos | Model | Restricciones | Permisos | Estadísticas | Disparadores | Flashback | Dependencias | Detalles | Particiones | Índices | SQL

Acciones...

	CONSTRAINT_NAME	CONSTRAINT_TYPE	SEARCH_CONDITION	R_OWNER	R_TABLE_NAME	R_CONSTRAINT_NAME	DELETE_RULE	
1	SYS_C0051478	Primary_Key	(null)	(null)	(null)	(null)	(null)	E

A diferencia, cuando lo creamos con el wizard se crean como sigue:

	CONSTRAINT_NAME	CONSTRAINT_TYPE	SEARCH_CONDITION
1	CONTINENTE_PK	Primary_Key	(null)
2	SYS_C0018703	Check	"CODIGOCONTINENTE" IS NOT NULL

Para contemplar las reglas exigidas, debemos indicar si las columnas son no nulas o no. Lo relevante que el primary key debe ser no nulo.

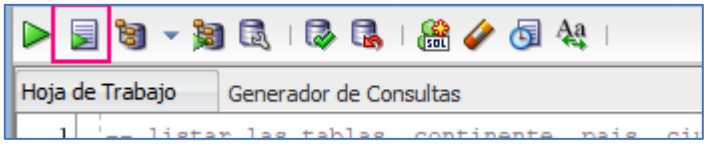
Si deseamos crear el modelo del Ejemplo del zoo con la segunda alternativa. Se tiene lo siguiente:

Todas las tablas del modelo físico del zoo sería lo siguiente:

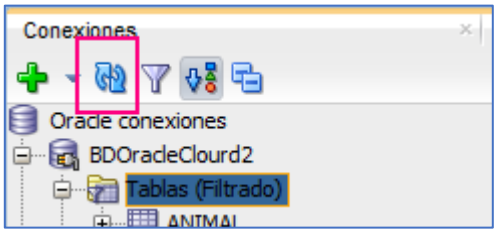
```
CREATE TABLE continente
(
  CODIGOCONTINENTE  VARCHAR2(3 BYTE) PRIMARY KEY,
  NOMBRECONTINENTE VARCHAR2(100 BYTE)
);

CREATE TABLE pais
(
  CODIGOPAIS  VARCHAR2(3 BYTE),
  NOMBREPAIS  VARCHAR2(100 BYTE),
  CODIGOCONTINENTE  VARCHAR2(3 BYTE),
  PRIMARY KEY (CODIGOPAIS)
);

CREATE TABLE ciudad
(
  CODIGOCIUDAD  VARCHAR2(3 BYTE),
  NOMBRECIUDAD  VARCHAR2(100 BYTE),
  CODIGOPAIS  VARCHAR2(3 BYTE),
  PRIMARY KEY (CODIGOCIUDAD)
);

CREATE TABLE zoo
(
  CODIGOZOO  VARCHAR2(3 BYTE),
  NOMBREZOO  VARCHAR2(100 BYTE),
  TAMANNO    VARCHAR2(100 BYTE),
  PRESUPUESTO  NUMBER,
  CODIGOCIUDAD  VARCHAR2(3 BYTE),
  PRIMARY KEY (CODIGOZOO)
);
```

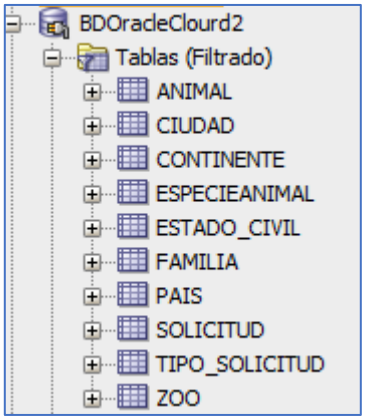
```
CREATE TABLE animal
(
  NUMEROIDENTIFICACION  VARCHAR2(3 BYTE),
  SEXO  CHAR(1 BYTE),
  ANNO  VARCHAR2(4 BYTE),
  CODIGOZOO  VARCHAR2(3 BYTE),
  CODIGOESPECIEANIMAL  VARCHAR2(3 BYTE),
  CODIGOPAIS  VARCHAR2(3 BYTE),
  PRIMARY KEY (NUMEROIDENTIFICACION)
);

CREATE TABLE especieanimal
(
  CODIGOESPECIEANIMAL  VARCHAR2(3 BYTE),
  NOMBREESPECIEANIMAL  VARCHAR2(100 BYTE),
  CODIGOFAMILIA  VARCHAR2(3 BYTE),
  PRIMARY KEY (CODIGOESPECIEANIMAL)
);



CREATE TABLE familia
(
  CODIGOFAMILIA  VARCHAR2(3 BYTE),
  NOMBREFAMILIA VARCHAR2(100 BYTE),
  PRIMARY KEY (CODIGOFAMILIA)
);
```

Para ejecutar este script anterior, debes presionar el icono remarcado en rosado. Ten presente dejar el cursor al inicio del archivo script



Refresca las tablas, para que aparezcan las tablas. Ubica el foco en “Tablas” (color azul) y luego presiona el icono enmarcado en rosado.





Si deseas crear nuevamente las tablas, elimina las tablas con las siguientes instrucciones:

```
drop table NOMBRETABLA;
```

Ejemplo;

```
drop table pais;
drop table continente;
drop table ciudad;
drop table animal;
drop table familia;
drop table especieanimal;
drop table zoo;
```

No tendrás problemas con la ejecución de la eliminación ya que aún las tablas no poseen llaves ajenas.

Resultado:

Table PAIS borrado.

Table CONTINENTE borrado.

Table CIUDAD borrado.

Table FAMILIA borrado.

Table ESPECIEANIMAL borrado.

Table ZOO borrado.

Al crear nuevamente las tablas, les agregaremos llaves ajenas. Para ello, debemos alterar cada tabla a través de script para asociarle las llaves ajenas.

La estructura del script es:

```
ALTER TABLE PAIS ADD (
  CONSTRAINT PAIS_CONTINENTE_FK
 FOREIGN KEY (CODIGOCONTINENTE)
 REFERENCES CONTINENTE (CODIGOCONTINENTE));
```

Nombre del FK a crear en al tabla Pais

columna de la tabla pais, asociada al FK

Tabla asociada a la tabla país

Llave primaria de la tabla continente

A continuación, el script de creación de las llaves ajenas es:

```
ALTER TABLE PAIS ADD (
  CONSTRAINT PAIS_CONTINENTE_FK
 FOREIGN KEY (CODIGOCONTINENTE)
 REFERENCES CONTINENTE (CODIGOCONTINENTE));

 ALTER TABLE CIUDAD ADD (
  CONSTRAINT CIUDAD_PAIS_FK
 FOREIGN KEY (CODIGOCIUDAD)
 REFERENCES PAIS (CODIGOPAIS));

 ALTER TABLE ZOO ADD (
  CONSTRAINT ZOO_CIUDAD_FK
 FOREIGN KEY (CODIGOCIUDAD)
 REFERENCES CIUDAD (CODIGOCIUDAD));

 ALTER TABLE ANIMAL ADD (
  CONSTRAINT ANIMAL_ZOO_FK
 FOREIGN KEY (CODIGOZOO)
 REFERENCES ZOO (CODIGOZOO));

 ALTER TABLE ANIMAL ADD (
  CONSTRAINT ANIMAL_ESPECIEANIMAL_FK
 FOREIGN KEY (CODIGOESPECIEANIMAL)
 REFERENCES ESPECIEANIMAL (CODIGOESPECIEANIMAL));

 ALTER TABLE ANIMAL ADD (
 CONSTRAINT ANIMAL_PAIS_FK
 FOREIGN KEY (CODIGOPAIS)
 REFERENCES PAIS (CODIGOPAIS));

ALTER TABLE ESPECIEANIMAL ADD (
 CONSTRAINT ESPECIEANIMAL_FAMILIA_FK
FOREIGN KEY (CODIGOFAMILIA)
REFERENCES FAMILIA (CODIGOFAMILIA));
```

Nota: como se altera la tabla para crear las llaves ajenas asimismo puedo alterar la tabla para crear las llaves primarias y así al crear la tabla solo con las columnas sin especificar la llave primaria. El alter table para crear primary key sería así.

```
alter table NOMBRETABLA
 add constraint NOMBRECONSTRAINT
 primary key (CAMPO,...);
```

Aplicando la sintaxi para la tabla país sería la instrucción como:

```
alter table PAIS
 add constraint PAIS_PK
 primary key (CODIGOPAIS);
```

A continuación, si eliminar nuevamente las tablas observarás que el orden de los drop importara. Ejemplo:

```
drop table pais;
drop table continente;
drop table ciudad;
drop table animal;
drop table familia;
drop table especieanimal;
drop table zoo;
```

Obtendrás, que más de una tabla no se puede eliminar, como se indica a través del siguiente mensaje.

```
Error que empieza en la línea: 143 del comando :
drop table pais
Informe de error -
ORA-02449: claves únicas/primarias en la tabla referidas por claves ajenas
02449. 00000 -  "unique/primary keys in table referenced by foreign keys"
*Cause:    An attempt was made to drop a table with unique or
           primary keys referenced by foreign keys in another table.
*Action:   Before performing the above operations the table, drop the
           foreign key constraints in other tables. You can see what
           constraints are referencing a table by issuing the following
           command:
           SELECT * FROM USER_CONSTRAINTS WHERE TABLE_NAME = "tabnam";
```

Piensa que para crear las tablas tuviste que preocuparte en la creación de las llaves ajenas, ya que estas necesitan que la llave primaria existiera de la tabla que hace relación. Para este caso de eliminación es lo mismo, debes validar que puedes borrar una tabla que nadie depende de ella.

Ejemplo: Prueba el siguiente orden

```sql
drop table animal;
drop table especieanimal;
drop table familia;
drop table zoo;
drop table ciudad;
drop table pais;
drop table continente;
```

No deberías tener problemas!!!

Existe otra alternativa para no tener problemas. Podrías eliminar las restricciones primero (llaves ajenas) y luego eliminar las tablas: Elimina todos los FK creados y luego las tablas.

Ejemplo:

```sql
ALTER TABLE PAIS
DROP CONSTRAINT PAIS_CONTINENTE_FK;

ALTER TABLE CIUDAD
DROP CONSTRAINT CIUDAD_PAIS_FK;

ALTER TABLE ZOO
DROP CONSTRAINT ZOO_CIUDAD_FK;

ALTER TABLE ANIMAL
DROP CONSTRAINT ANIMAL_ZOO_FK;

ALTER TABLE ANIMAL
DROP CONSTRAINT ANIMAL_ESPECIEANIMAL_FK;

ALTER TABLE ANIMAL
DROP CONSTRAINT ANIMAL_PAIS_FK;

ALTER TABLE ESPECIEANIMAL
DROP CONSTRAINT ESPECIEANIMAL_FAMILIA_FK ;
```

Y luego eliminamos las tablas sin respetar ningún orden……

```sql
drop table zoo;
drop table familia;
drop table pais;
drop table ciudad;
drop table especieanimal;
drop table continente;
drop table animal;
```

Al inicio del documento, vimos algunas alternativas de creación de tablas.

Operaciones con datos

- **<u>Ingresar datos:</u>**

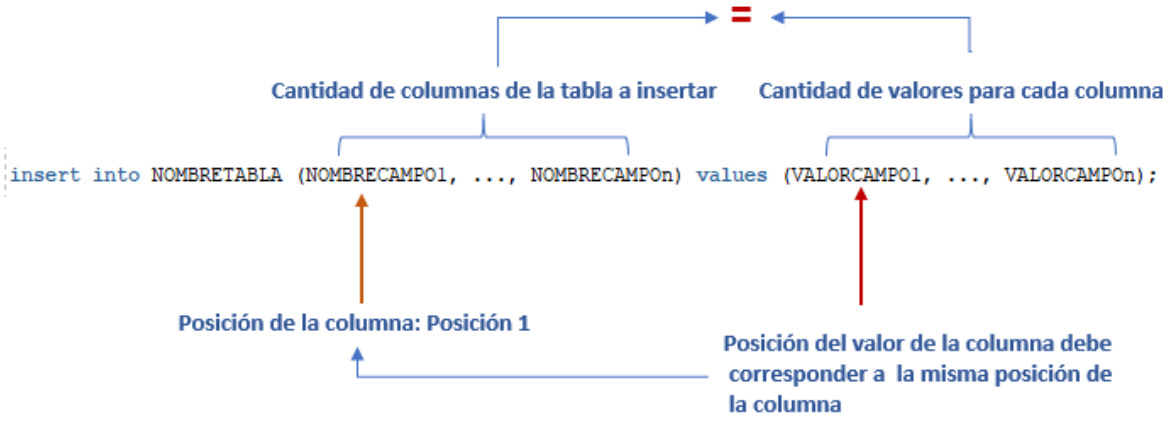
Un registro es una fila de la tabla que contiene los datos. Cada registro tiene un dato por cada columna (campo).

Al ingresar los datos de cada registro debe tenerse en cuenta **la cantidad y el orden de los campos.**

La sintaxis básica y general es la siguiente:

```
insert into NOMBRETABLA (NOMBRECAMPO1, ..., NOMBRECAMPOn) values (VALORCAMPO1, ..., VALORCAMPOn);
```

Usamos "insert into", luego el nombre de la tabla, detallamos los nombres de los campos entre paréntesis y separados por comas y luego de la cláusula "values" colocamos los valores para cada campo, también entre paréntesis y separados por comas. A continuación,



Para probar el ingreso de datos. Ingresaremos datos a la tabla continente. Recordemos la estructura de la tabla continente. La tabla continente tiene dos columnas, código continente y nombrecontinente, ambos tipo texto (varchar2), el primero no nulo y el segundo puede ser nulo.

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	CODIGOCONTINENTE	VARCHAR2(3 BYTE)	No	(null)	1	(null)
2	NOMBRECONTINENTE	VARCHAR2(100 BYTE)	Yes	(null)	2	(null)

Si sabemos los continentes del mundo, ya tenemos los datos que deberemos indicar en la sintaxi de la instrucción sql para insertar datos. En este ejemplo le indicamos valores a todas las columnas de la tabla. No obstante, como mínimo debemos insertar todas aquellas columnas que no pueden ser nulos (NULLABLE) como se indica en la imagen anterior.

```sql
insert into continente (codigocontinente, nombrecontinente) values ('001','America del Norte');
insert into continente (codigocontinente, nombrecontinente) values ('002','America del Sur');
insert into continente (codigocontinente, nombrecontinente) values ('003','Oceanía');
insert into continente (codigocontinente, nombrecontinente) values ('004','Africa');
insert into continente (codigocontinente, nombrecontinente) values ('005','Antartida');
insert into continente (codigocontinente, nombrecontinente) values ('006','Asia');
insert into continente (codigocontinente, nombrecontinente) values ('007','Europa');

commit;
```

De la imagen anterior, podemos darnos cuenta que si los datos a insertar son de tipo de texto los valores deben ir entre comillas simples.

La instrucción al final de la imagen anterior "**commit;**" es una instrucción sql que permite confirmar el guardado en la base de datos. Si no indicamos esta instrucción, podríamos deshacer los cambios en la base de datos a través de la instrucción **rollback;**

Finalmente, la tabla resulta.

	CODIGOCONTINENTE	NOMBRECONTINENTE
1	001	America del Norte
2	002	America del Sur
3	003	Oceanía
4	004	Africa
5	005	Antartida
6	006	Asia
7	007	Europa

Como les mencione, podría insertar como mínimo aquellos valores de columnas no nulos. En este caso de la tabla continente, podríamos solo insertar el código.

```sql
insert into continente (codigocontinente) values ('010');
commit;
```

	CODIGOCONTINENTE	NOMBRECONTINENTE
1	001	America del Norte
2	002	America del Sur
3	003	Oceanía
4	004	Africa
5	005	Antartida
6	006	Asia
7	007	Europa
8	010	(null)

- **Actualizar datos:**

Decimos que actualizamos un registro cuando modificamos alguno de sus valores.

Para modificar uno o varios datos de uno o varios registros utilizamos "update" (actualizar).

Sintaxis básica:

```
update NOMBRETABLA set COLUMNA=NUEVOVALOR;

O

update NOMBRETABLA set COLUMNA=NUEVOVALOR, COLUMNA=NUEVOVALOR, .... ;
```

Utilizamos "update" junto al nombre de la tabla y "set" junto con el campo a modificar y su nuevo valor.

Ejemplo: tabla antes de la actualización

	CODIGOCONTINENTE	NOMBRECONTINENTE
1	001	America del Norte
2	002	America del Sur
3	003	Oceanía
4	004	Africa
5	005	Antartida
6	006	Asia
7	007	Europa

Ejercicio: actualizar la tabla continente modificando el **valor** de la **columna nombrecontinente** a '**continente'**. En lenguaje sql tenemos:

```
update continente set nombrecontinente='continente'
```

El resultado de la actualización es:

	CODIGOCONTINENTE	NOMBRECONTINENTE
1	001	continente
2	002	continente
3	003	continente
4	004	continente
5	005	continente
6	006	continente
7	007	continente

Si lo observas en detalle modifico todos los registros de la tabla.

Modificar registros evaluando condiciones

Veamos otro ejemplo, la tabla persona que a continuación se muestra

ID	NOMBRE	APELLIDO	EDAD	SEXO
51	Nombre1	Apellido1	12	F
52	Nombre2	Apellido2	13	M
53	Nombre3	Apellido3	14	F
54	Nombre4	Apellido4	15	M
55	Nombre5	Apellido5	16	M
56	Nombre6	Apellido6	17	F

Si deseo modificar el registro (enmarcado en rosado) cuyo ID es 54, los nuevos valores son: juanito en la columna Nombre, muñoz en la columna Apellido, los otros datos asociados al registro de las columnas no se modifican.

ID	NOMBRE	APELLIDO	EDAD	SEXO
51	Nombre1	Apellido1	12	F
52	Nombre2	Apellido2	13	M
53	Nombre3	Apellido3	14	F
54	Nombre4	Apellido4	15	M
55	Nombre5	Apellido5	16	M
56	Nombre6	Apellido6	17	F

La instrucción será: actualizar la tabla persona, los valores nuevos son nombre='juanito' y apellido='muñoz' donde id =54. La condición parte en la cláusula where.

La lectura es: Actualiza el registro cuyo ID es 54 y modifica las columnas nombre y apellido. Los valores son juanito y muñoz respectivamente.

```
update persona set nombre='juanito', apellido='muñoz'
where id=54;
```

Recuerda que después de está instrucción confirma las modificaciones con una instrucción **commit**.

	ID	NOMBRE	APE...	EDAD	SEXO
1	51	Nombre1	Apellido1	12	F
2	52	Nombre2	Apellido2	13	M
3	53	Nombre3	Apellido3	14	F
4	54	juanito	muñoz	15	M
5	55	Nombre5	Apellido5	16	M
6	56	Nombre6	Apellido6	17	F

La clausula **where** permite evaluar distintas condiciones en base a los valores de las columnas. Volveremos a retomar la actualización de registros cuando pasemos las consultas a las tablas.